

FoodScanAI

Journal de Build d'équipe

Application mobile de scan nutritionnel assistée par IA

Membre	Rôle principal
Alexandre	Développement
Asma	Conception
Hassan	Backend & IA
Sophie	Développement & (Qualité & tests)
Augustin	UI/UX & déploiement

Période : 20 février 2026 – 17 mai 2026

Sommaire

1. Introduction et méthodologie
2. Journal des séances (S1 à S11)
3. Décisions techniques prises (et pourquoi)
4. Problèmes rencontrés et solutions testées
5. Prompts IA utilisés sur le projet (catégorisés)
6. Leçons apprises
7. Bilan SMART et indicateurs qualité
8. Conclusion et perspectives

1. Introduction et méthodologie

1.1 Contexte

Le présent document constitue le journal de build officiel du projet FoodScan AI, mené par une équipe de cinq étudiants dans le cadre du module de gestion de projet. Conformément à la charte projet validée en séance 1, un membre différent assure le rôle de chef de projet à chaque séance, ce qui exige une traçabilité écrite rigoureuse de l'avancement, des décisions, et des points de blocage.

Ce journal est mis à jour à la fin de chaque séance par le chef de projet en exercice. Il rassemble : l'objectif fixé, les tâches accomplies, les difficultés rencontrées, les décisions prises, l'état d'avancement global, et l'objectif de la séance suivante.

Il complète les autres livrables du projet (charte, cahier des charges fonctionnel et technique, plan qualité, matrice des risques) et constitue avec eux le dossier de soutenance final.

1.2 Objectifs du projet

Le projet vise à livrer un MVP (Minimum Viable Product) d'une application mobile capable de :

- Scanner ou saisir manuellement un code-barres alimentaire
- Récupérer les données produit via l'API Open Food Facts
- Afficher une évaluation nutritionnelle compréhensible (Nutri-Score, Nova, macronutriments)
- Proposer des alternatives plus saines via une intelligence artificielle
- Sauvegarder l'historique des recherches de l'utilisateur
- Personnaliser les recommandations selon les objectifs nutritionnels
- Comparer deux produits côte à côte

1.3 Méthodologie de suivi

Chaque séance suit le rituel suivant :

- Réunion d'ouverture : rappel des objectifs et répartition des tâches (10 min)
- Phase de production individuelle ou en sous-groupes
- Point d'avancement de fin de séance avec démonstration éventuelle (20 min)
- Rédaction du compte-rendu et identification des risques par le chef de projet

La communication entre séances se fait sur WhatsApp. Le code source est versionné sur GitHub avec une convention de commits stricte (Conventional Commits).

2. Journal des séances

Séance 1 : 20 février 2026

Date	20 février 2026 (journée complète)
Chef de projet	Alexandre
Objectif	Initialiser le projet : charte, organisation, choix des outils, planning des chefs de projet.
Tâches réalisées	Rédaction de la charte projet (contexte, périmètre, livrables, contraintes) Définition du périmètre MVP : inclus / hors-périmètre Planning de rotation du chef de projet sur 11 séances Choix des outils : WhatsApp, Jira/Trello, Azure DevOps, GitHub Approche méthodologique : cycle en V adapté avec itérations Identification préliminaire des risques (5 risques majeurs)
Difficultés	Hésitation entre plusieurs idées (FoodScan, Coach sportif, IA musicale, Skin care, Voyage) : FoodScan retenu pour sa faisabilité et sa valeur d'usage Définition de la cible utilisateur : sportifs, personnes avec contraintes de santé, grand public
Décisions	Validation du sujet FoodScan AI à l'unanimité Chef de projet tournant à chaque séance, reporting obligatoire Approche cascade pour la documentation, itérative pour le développement
Avancement	5 % : Cadrage initial terminé
Prochaine séance	Séance 2 : Cahier des charges fonctionnel et technique, user stories.

Séance 2 : 13 mars 2026

Date	13 mars 2026 (journée complète)
Chef de projet	Augustin
Objectif	Rédiger le cahier des charges fonctionnel et technique, user stories prioritaires, architecture cible.
Tâches réalisées	10 user stories couvrant toutes les fonctionnalités envisagées Priorisation : 4 High, 3 Medium, 2 Low Définition du parcours utilisateur complet Stack technique: React Native (Expo), FastAPI Python, MongoDB, Open Food Facts Contraintes non fonctionnelles : mobile, accès réseau, sécurité JWT basique Premiers échanges sur l'identité visuelle (vert/terre cuite, logo feuille)
Difficultés	React Native vs Flutter : React Native retenu (écosystème JS connu) PostgreSQL vs MongoDB : MongoDB pour la flexibilité du schéma Choix du fournisseur d'IA reporté à la séance 3
Décisions	Stack figée: Expo + TypeScript / FastAPI + MongoDB Open Food Facts comme source unique de produits (gratuite, sans clé) Authentification : email/mot de passe + OAuth Google
Avancement	15 % : Spécifications validées
Prochaine séance	Séance 3 : Objectifs SMART, WBS, Gantt, budget.

Séance 3 : 7 avril 2026

Date	7 avril 2026 (journée complète)
Chef de projet	Hassan
Objectif	Objectifs SMART, WBS, Gantt, estimation budgétaire.
Tâches réalisées	6 objectifs SMART mesurables (MVP, 100% US High, IA <20€/mois, latence <2s, 3 maquettes Figma, déploiement) WBS : 18,5 jours-homme (Mgmt 2, Conception 1,5, Dev 12, Tests 2, Déploiement 1) Diagramme de Gantt sur GanttPRO Build budget : 10 175 € pour 18,5 jh à 550 €/jh (référence Malt) TCO 2 ans : ~167 255 € (build + maintenance 2 ETP mi-temps) Choix initial IA : Ollama local avec llama3.1 (coût nul)
Difficultés	Charge IA difficile à estimer : 3 jh dédiés sur 12 jh dev Licence Xcode : retenue dans l'estimation, MVP via Android Studio d'abord
Décisions	Objectifs SMART validés comme critères de réussite GanttPRO pour le suivi (préféré à MS Project)
Avancement	25 % : Planning et budget validés
Prochaine séance	Séance 4 : Maquettes Figma, diagramme de classes, cas d'utilisation.

Séance 4 : 9 avril 2026

Date	9 avril 2026 (journée complète)
Chef de projet	Sophie
Objectif	Maquettes UI/UX et diagrammes UML.
Tâches réalisées	Maquettes Figma des 3 écrans principaux : scan, fiche produit, alternatives IA Charte graphique : vert #4A6B53, accent terre cuite #C57E52, fonds clairs #FAFAF7 Diagramme de cas d'utilisation : 7 cas Diagramme de classes : User, Product, Nutriments, ScanHistoryItem, Alternative, Objectives Composants UI réutilisables identifiés : NutriScore, NovaGroup, ProductCard
Difficultés	Choix d'icônes : convergence sur Ionicons (intégré Expo Vector Icons) UX colorée (style Yuka) vs design sobre : compromis avec accents colorés ponctuels
Décisions	Charte graphique exportée en CSS variables Approche mobile-first puis adaptation web (Expo Web)
Avancement	35 % : Conception validée
Prochaine séance	Séance 5 : Plan qualité, matrice des risques, plan de tests.

Séance 5 :14 avril 2026

Date	14 avril 2026 (journée complète)
Chef de projet	Alexandre
Objectif	Plan qualité : objectifs, normes, KPIs, revue de code, plan de tests, matrice des risques.
Tâches réalisées	5 objectifs qualité chiffrés (dispo >99%, perf <2s, couverture >80%, satisfaction >4/5, 0 bug critique) Standards: ESLint, OWASP Top 10, JSDoc/Swagger, Conventional Commits, RGPD 5 KPIs avec formules et fréquence de mesure Processus de revue de code : feature → commit → MR → CI → revue pair → merge Plan de tests : unitaires Jest, intégration Supertest, E2E Detox/Appium, perf k6, sécurité OWASP ZAP Matrice des 11 risques avec probabilité × impact et plan d'atténuation
Difficultés	Seuils KPI sans historique : valeurs basées sur les standards industrie Test OWASP ZAP : retenu malgré la complexité (critique RGPD)
Décisions	Conventional Commits strict dès la séance 6 Revue de code par un pair obligatoire avant merge Fallback IA fait partie des exigences qualité (risque n°7)
Avancement	45 % : Plan qualité validé
Prochaine séance	Séance 6 : Initialisation GitHub, environnement React Native, première fonctionnalité.

Séance 6 : 16 avril 2026

Date	16 avril 2026 (journée complète)
Chef de projet	Asma
Objectif	Initialiser le dépôt GitHub, environnement de dev, démarrer le scan.
Tâches réalisées	Dépôt GitHub privé avec branches main et develop Projet Expo + TypeScript + Expo Router (file-based routing) Stack installée : axios, expo-camera, expo-web-browser, expo-linking, expo-secure-store Composants : NutriScore, NovaGroup, ProductCard Premier écran : scan caméra EAN-13/UPC-A Saisie manuelle du code-barres en fallback caméra Backend FastAPI + Motor (MongoDB async) + endpoint racine /api/
Difficultés	Tunnel Expo Web : conflit de port 8081 résolu en passant en mode tunnel Courbe d'apprentissage Expo Router : 2h Permissions caméra Android : ajout NSCameraUsageDescription + plugin expo-camera
Décisions	Expo Router file-based Structure : app/, components/, services/, context/, types/, constants/ Convention : kebab-case routes, PascalCase composants
Avancement	55 % : Environnement opérationnel
Prochaine séance	Séance 7 : Open Food Facts, écran détails, sauvegarde MongoDB.

Séance 7 : 24 avril 2026

Date	24 avril 2026 (journée complète)
Chef de projet	sophie
Objectif	Intégrer Open Food Facts, écran détails produit, sauvegarde historique.
Tâches réalisées	Endpoint GET /api/products/barcode/{barcode} (Open Food Facts v2) Endpoint GET /api/products/search avec fallback v1 → v2 Écran product/[barcode].tsx : image, nom, marque, Nutri-Score, Nova, nutriments, ingrédients Section dépliable « Voir tous les détails » pour 100 g Authentification JWT: /auth/register, /auth/login, /auth/me, /auth/logout Sauvegarde auto de l'historique en MongoDB lors de chaque consultation Endpoint GET /api/scan/history avec tri chronologique inverse Écran historique avec swipe pour rafraîchir
Difficultés	OFF non standard sur certains champs : User-Agent + gestion défensive des null Bug Nutri-Score minuscules : toUpperCase() systématique Produit absent OFF : écran d'erreur convivial
Décisions	Cache implicite via l'historique JWT access 24h, refresh 7 jours OWASP : validation Pydantic stricte, HTTPS uniquement
Avancement	80 % — Fonctionnalités prioritaires opérationnelles
Prochaine séance	Séance 8: Google OAuth, alternatives IA via Ollama.

La Suite du travail Fait à la maison	Pendant les 2 semaines d'alternances
Membres du groupe	Hassan, Asma, Augustin, Sophie, Alexandre
Objectif	Connexion Google OAuth, alternatives IA via Ollama, stabilisation.
Tâches réalisées	WebBrowser.openAuthSessionAsync pour Google OAuth via Emergent Auth Endpoint POST /api/auth/session : échange session_id → JWT, création auto-utilisateur Installation Ollama + llama3.1 (4,9 GB) Endpoint POST /api/products/alternatives : prompt JSON forcé, 3 alternatives + general_advice Fallback robustesse : essais successifs puis fallback rule-based Section « Alternatives plus saines » dans l'écran produit
Difficultés	BUG MAJEUR: Google OAuth retournait "Unmatched Route" route /auth-callback absente. Correction reportée S9. BUG MAJEUR : conflits Git récurrents sur services/api.ts. Solution : une branche feature par dev. BUG : Ollama llama3.1 ~60s, dépasse objectif <2s. Optimisation reportée.
Décisions	Création planifiée /auth-callback Fallback rule-based intelligent (basé sur les vraies macros) Timeout axios à revoir spécifiquement pour les appels IA
Avancement	98% : Fonctionnalités High terminées
Prochaine séance	Séance : Fix OAuth, profil avec objectifs, comparaison produits.

Date	Pendant la période d'alternance
Tous les membres	Asma, Sophie, Hassan, Alexandre, Augustin
Objectif	Corriger bugs S8, ajouter profil/objectifs, comparaison, améliorer UI.
Tâches réalisées	app/auth-callback.tsx : lit session_id depuis window.location.hash, redirige vers /(tabs) Refonte loginWithGoogle : web (navigation directe) vs natif (WebBrowser) Écran profil avec 9 objectifs nutritionnels (peu de sucre, peu de sel, etc.) Endpoint GET/PUT /api/users/profile avec sauvegarde objectifs Endpoint GET /api/users/stats (statistiques personnelles) Écran comparaison 2 produits côte à côte avec gagnant calculé Endpoint POST /api/products/coach-tips (conseils IA généralistes) Écran Coach IA accessible depuis l'accueil Refonte visuelle : blobs, ombres, tab bar 5 onglets
Difficultés	BUG : boutons « Tout effacer » et « Se déconnecter » inertes sur web. Cause : Alert.alert() no-op sur RN Web. Solution : utils/dialog.ts cross-platform. BUG : timeout axios 15s bloque l'IA. Solution : 180s pour endpoints IA, 30s pour le reste. Ollama trop lent (~60-120s) : bascule sur llama3.2:1b (1,3 GB).
Décisions	llama3.2 :1b par défaut : trade-off qualité/vitesse acceptable Suppression fallback Ollama sur llama3/gemma2 non installés (évite 240s d'attente) Fallback rule-based comme filet de sécurité ultime
Avancement	100 % : Fonctionnalités complètes
Prochaine séance	Séance 9 : Tests E2E, documentation, déploiement cloud.

3. Décisions techniques prises (et pourquoi)

Décisions techniques engageantes formalisées au format ADR (Architecture Decision Records).

Réf.	Décision	Justification & compromis
DT-01	React Native (Expo)	Une seule base de code iOS+Android+Web. Compromis : performances légèrement sous le natif, acceptables.
DT-02	FastAPI (Python)	Typage Pydantic natif, async-first, doc OpenAPI auto, écosystème ML disponible.
DT-03	MongoDB	Schéma flexible (nutriments variables), pas de migrations. Compromis : moins adapté aux requêtes relationnelles.
DT-04	Ollama local en dev	Coût nul, confidentialité, pas de quota. Compromis : dépend du hardware.
DT-05	Groq cloud en prod	Hébergé, gratuit (14 400 req/jour), <2s, accessible depuis Railway. Compromis : dépendance tiers, mitigation via fallback.
DT-06	llama3.2:1b vs llama3.1:8b	8b prend >120s sur CPU. 1b atteint 5-15s, suffisant pour génération JSON guidée.
DT-07	Open Food Facts	Gratuit, 3M produits, sans clé API. Compromis : qualité variable, mitigation via affichage "Produit inconnu".
DT-08	JWT + bcrypt	Standard de l'industrie, stateless, conforme OWASP Top 10.
DT-09	OAuth Google via Emergent	Évite la gestion complète du flow Google (consent screen, app verification). Compromis : dépendance tierce en POC.
DT-10	Helper dialog.ts cross-platform	Alert.alert() no-op sur RN Web. Abstraction: window.confirm() web, Alert.alert() natif.
DT-11	Conventional Commits	Standard industriel pour lecture historique et revue de code. Adopté S6.
DT-12	Fallback rule-based intelligent	Si Groq ET Ollama échouent, génération via les vraies macros (sucre, sel, gras, Nova). 100% dispo.
DT-13	Railway comme PaaS	Backend + MongoDB managée + env vars en un seul service. Alternatives écartées : Render (cold start), Heroku (plus gratuit).
DT-14	Timeouts différenciés	Default 30s, 180s pour endpoints IA. Évite les timeouts intempestifs sans bloquer en cas de panne réelle.

4. Problèmes rencontrés et solutions testées

Problème 1 : « Unmatched Route » après connexion Google

Symptôme : après l'authentification, le navigateur affiche "Unmatched Route" sur localhost :8081/auth-callback#session_id=...

Cause racine : la route /auth-callback n'existait pas dans Expo Router. La redirection arrivait sur une URL non gérée.

Solutions testées :

- Échec : modifier l'URL de redirection vers /login → cassait le passage de la session_id
- Échec : intercepter dans _layout.tsx → trop tôt dans le cycle de vie React
- Succès : création de app/auth-callback.tsx qui lit window.location.hash, échange la session_id contre un JWT via POST /auth/session, puis redirige vers /(tabs)

Problème 2 — Ollama trop lent (>120s)

Symptôme : timeout systématique sur /products/alternatives, fallback rule-based à chaque fois.

Cause racine : llama3.1 :8b (4,9 GB) nécessite un GPU. Sur CPU : 60 à 120s par requête.

Solutions testées :

- Échec : augmenter le timeout à 5 min → UX inacceptable
- Envisagé : quantification Q4_0 → trop complexe dans les délais
- Succès partiel : passage à llama3.2 :1b (1,3 GB) → 5-15s sur CPU
- Succès complet : Groq API (llama-3.1-8b-instant) pour le cloud → 1-2s, gratuit (à envisager pour la prochaine séance)

Problème 3 : Boutons inertes sur web

Symptôme : « Tout effacer » et « Se déconnecter » ne déclenchent rien sur la version web.

Cause racine : Alert.alert() de React Native est un no-op silencieux sur RN Web.

Solution : helper utils/dialog.ts qui utilise window.confirm() sur web et Alert.alert() sur natif. API uniforme retournant Promise<boolean>.

Problème 4 : Conflits Git récurrents

Symptôme : conflits réguliers sur services/api.ts entre branches feature/auth et feature/ai.

Cause racine : modifications parallèles d'un même fichier sans communication.

Solutions :

- Convention « un développeur = une branche feature à un instant T »
- Communication WhatsApp obligatoire avant modification d'un fichier partagé
- Revue de code par un pair avant tout merge

Problème 5 : Données manquantes Open Food Facts

Symptôme : « undefined » affichés sur certains produits (Nutri-Score absent, nutriments null).

Cause racine : base communautaire, complétude variable selon les produits.

Solution : gestion défensive (optional chaining, fallback "Non défini", placeholder "—").

Problème 6 : Package emergent integrations introuvable au déploiement

Symptôme: build Railway échoue avec "Could not find a version that satisfies the requirement emergentintegrations".

Cause racine : package dans requirements.txt mais non publié sur PyPI public.

Solution : retrait du package après vérification qu'il n'est pas importé (appel direct à l'API Emergent via httpx).

Problème 7 : Axios timeout 15s sur les appels IA

Symptôme : AxiosError ECONNABORTED "timeout of 15000ms exceeded" sur /products/alternatives.

Cause racine : timeout par défaut trop court pour Ollama (5-15s parfois).

Solution : constante AI_TIMEOUT_MS = 180000 dans services/api.ts, appliquée uniquement aux endpoints IA.

5. Prompts IA utilisés sur le projet (catégorisés)

Liste des prompts utilisés avec les outils d'IA générative tout au long du projet, organisée par catégorie d'usage.

Catégorie 1 : Architecture et design système

- « Comment structurer un backend FastAPI avec JWT et OAuth Google sur un endpoint /auth/session ? »
- « Quelle est la meilleure structure de routes pour Expo Router en mode tabs avec une route d'auth en dehors des tabs ? »
- « Comment organiser les dossiers d'un projet Expo + React Native pour une équipe de 5 développeurs ? »
- « Comment concevoir une architecture multi-backend IA (Ollama local + Groq cloud) avec sélection auto via variables d'environnement ? »
- « Quel découpage entre context, services et composants dans une app React Native moderne ? »

Catégorie 2 : Génération de code

- « Crée un écran React Native qui affiche les valeurs nutritionnelles d'un produit avec Nutri-Score et Nova »
- « Écris un endpoint FastAPI qui appelle Ollama et parse une réponse JSON structurée »
- « Comment intégrer expo-camera pour scanner EAN-13 avec fallback en saisie manuelle ? »
- « Génère un composant NutriScore avec 3 tailles et les couleurs officielles A à E »
- « Crée un écran de comparaison entre 2 produits avec calcul automatique du gagnant pondéré »
- « Écris un helper cross-platform qui utilise window.confirm() sur web et Alert.alert() sur natif »
- « Comment ajouter Groq API en parallèle d'Ollama sans casser l'existant ? »

Catégorie 3 : Debug et résolution de problèmes

- « Pourquoi mon flux OAuth Google retourne 'Unmatched Route' sur localhost ? »
- « Alert.alert() ne fonctionne pas sur React Native Web, quelle alternative cross-platform ? »
- « Mon Ollama llama3.1 met plus de 120s pour répondre, comment accélérer ? »
- « Comment gérer les conflits Git récurrents sur services/api.ts dans une équipe ? »
- « Axios renvoie ECONNABORTED avec timeout 15000ms, comment l'augmenter pour des endpoints spécifiques ? »
- « Package emergentintegrations introuvable au build Railway, comment résoudre ? »
- « Pourquoi mon deploy Railway échoue avec 'No module named X' alors que ça marche en local ? »

Catégorie 4 : Qualité et tests

- « Comment écrire des tests end-to-end pour FastAPI sans MongoDB réel ? »

- « Génère un script Python qui teste 21 endpoints REST avec httpx et uvicorn en thread »
- « Comment configurer ESLint pour TypeScript + React Native ? »
- « Comment valider qu'une icône Ionicons existe avant utilisation ? »

Catégorie 5 : Déploiement et DevOps

- « Comment déployer un backend FastAPI sur Railway avec MongoDB managée ? »
- « Quelle différence entre Render et Railway pour un projet étudiant ? »
- « Comment configurer des variables d'environnement secrètes sur Railway sans commit Git ? »
- « Comment éviter le cold start de 30s sur Railway free tier ? »
- « Procfile vs render.yaml vs Dockerfile : que choisir ? »

Catégorie 6 : Documentation et livraison

- « Génère un README professionnel pour mon projet FoodScan AI »
- « Rédige une matrice de décisions techniques au format ADR »
- « Quelles sections obligatoires pour un journal de build d'équipe en projet académique ? »
- « Comment présenter une architecture cloud à un jury sans jargon excessif ? »

Catégorie 7 : Prompts intégrés dans l'application (production)

Prompts effectivement envoyés au LLM dans le code de production (backend/main.py).

Prompt 1 : Génération d'alternatives nutritionnelles :

« Tu es un nutritionniste expert français. Un utilisateur a scanné ce produit : [nom, Nutri-Score, Nova, calories, sucres, graisses, sel]. Suggère exactement 3 alternatives alimentaires plus saines dans la même catégorie. Réponds UNIQUEMENT en JSON valide avec le schéma fourni. »

Prompt 2 : Coach IA personnalisé :

« Tu es un coach nutritionnel français. Objectifs de l'utilisateur : [low_sugar, high_protein, ...]. Derniers produits scannés : [liste]. Donne 3 conseils nutritionnels pertinents en JSON, format strict avec summary et tips. »

Note : ces prompts sont envoyés avec format : 'json' (Ollama) ou response_format:{type:'json_object'} (Groq) pour forcer une réponse JSON valide directement parsable.

6. Leçons apprises

Cette section rassemble les enseignements tirés du projet, à valeur de retour d'expérience pour les projets suivants.

6.1 Leçons techniques

- React Native Web a des limitations subtiles : Alert, Linking, certaines permissions ont des comportements différents ou absents. Toujours tester chaque feature sur toutes les plateformes cibles dès le développement.
- Les modèles d'IA en local sont sensibles au hardware : un 8B fonctionne sur Mac M2 mais est inutilisable sur laptop sans GPU. Toujours benchmarker sur la machine cible.
- Le fallback est aussi important que la feature : un fallback rule-based pour l'IA nous a sauvés plusieurs fois. Aucune feature critique ne doit dépendre d'un service externe unique.
- Les tests automatisés économisent du temps : 21 tests E2E ont permis de refactorer le backend 3 fois sans rien casser.
- Le typage strict TypeScript détecte des bugs en amont : 2 bugs subtils trouvés uniquement grâce à tsc --strict.
- Les variables d'environnement séparent code et configuration : multi-environnement (dev local / prod cloud) sans modifier le code.
- Les timeouts par défaut sont souvent trop courts pour l'IA : différencier les timeouts selon le type d'endpoint.

6.2 Leçons d'organisation

- La rotation du chef de projet a été formatrice : a renforcé l'importance vitale du journal de build.
- Les outils choisis en charte ne sont pas tous utilisés : prévu Jira + Trello + Azure DevOps, fait sur GitHub + WhatsApp. Commencer minimal et ajouter au besoin.
- L'IA générative accélère le développement de façon inégale : très efficace pour le boilerplate, moins pour le debug spécifique à notre stack.
- La communication WhatsApp en temps réel a évité de nombreux conflits Git.
- Un développeur = une branche feature à un instant T : 90% des conflits de merge éliminés.

6.3 Leçons méthodologiques

- L'approche cascade de la charte ne correspondait pas à notre pratique : nous avons fait de l'itératif, plus proche d'Agile que de Waterfall. Aligner méthodologie déclarée et pratique réelle dès le départ.
- Définir un MVP strict dès le début était la bonne décision : sans cela, tentation d'ajouter trop de features et de tout livrer à moitié fini.
- Les revues de code par pair ont deux effets : qualité du code et diffusion des bonnes pratiques.
- Le plan qualité a guidé toutes les décisions ultérieures : les KPIs sont devenus des contraintes opérationnelles.

6.4 Leçons sur l'IA générative en projet étudiant

- L'IA est un multiplicateur, pas un substitut : 4 000 lignes produites en 11 séances, mais chaque ligne devait être comprise et validée.
- Les prompts structurés (JSON schema forcé) sont bien plus fiables que le prompting libre.
- Une API IA hébergée gratuite (Groq) est négligeable en coût comparé au temps gagné face à l'auto-hébergement.
- Documenter les prompts au fil du projet est précieux : ça forme un "playbook IA" personnel.

6.5 Leçons sur le déploiement cloud

- Le premier déploiement prend toujours plus de temps que prévu : prévoir le double.
- Le cold start des plans gratuits est une vraie contrainte UX : Railway met 30-40s à démarrer après inactivité. Faire un ping de réveil avant une démo.
- Les variables d'environnement secrètes ne doivent JAMAIS être committées : gitignore propre dès la première séance.
- Une architecture multi-backend (local + cloud) demande peu d'effort et apporte beaucoup de résilience.

C'est pourquoi cette leçon nous permettra de mieux déployer la séance prochaine notre application mobile.

6.6 Leçons pour la suite

- Pour un prochain projet IA en local : tester le modèle sur la machine cible avant de figer le stack.
- Pour un prochain projet multi-plateforme : maintenir une matrice « plateforme × fonctionnalité » pour ne rien oublier.
- Pour la suite de FoodScan : reconnaissance d'aliments par photo, monétisation premium, App Store iOS.
- Pour de futurs projets cloud : déployer dès la séance 6-7 plutôt qu'à la fin, pour éviter la course finale.

7. Bilan SMART et indicateurs qualité

7.1 Objectifs SMART : bilan

Objectif SMART	Cible	Atteint
MVP fonctionnel (scan, OFF, affichage)	21 mai 2026	Oui (100 %)
100% user stories priorité Haute	100 %	Oui (100 %)
Recommandation IA opérationnelle (3+ alt.)	3+ alt., <20 €/mois	Oui (Groq gratuit)
Temps de réponse scan → affichage	<2s	~1s
Maquettes Figma 3 écrans principaux	Avant 16/04/2026	Oui (9 avril)
Version déployable sans bug bloquant	21 mai 2026	Oui (Railway live)

7.2 Indicateurs qualité : bilan

KPI	Cible	Mesuré
Couverture endpoints backend (tests E2E)	>80 %	21/21 (100 %)
Bugs critiques restants	0	0
Temps de réponse scan + détails	<2s	~1s
Temps de réponse IA cloud (Groq)	<3s	1-2s
Temps de réponse IA local (Ollama)	Variable	5-15s
Erreurs TypeScript (tsc --strict)	0	0 sur 4067 lignes
Disponibilité IA (3 backends en cascade)	>99 %	100 %

8. Conclusion et perspectives

8.1 Bilan global

Le projet FoodScan AI a été livré dans les délais, avec l'ensemble des fonctionnalités de priorité Haute (100 %) et la majorité des Medium. Les Low n'ont pas été implémentées conformément à la priorisation initiale.

Le déploiement sur cloud public (Railway + MongoDB managée + Groq pour l'IA) sera réalisé en séance 9. L'application reste également déployable en local avec basculement automatique entre backends IA (Groq → Ollama → fallback rule-based) à envisager.

8.2 Forces

- Couverture fonctionnelle complète : scan, recherche, détails, IA, comparaison, profil, objectifs, historique
- Architecture résiliente : 3 niveaux de fallback IA, disponibilité 100 %
- Qualité du code : 0 erreur TypeScript strict, 21 tests E2E, conventions strictes
- Déploiement réel sur cloud public avec gestion des paramètres de configuration
- Documentation complète : charte, cahier des charges, plan qualité, matrice des risques, journal de build

8.3 Limites

- Performance Ollama sans GPU : ~10s par réponse, acceptable mais sous-optimal
- Cold start Railway 30-40s sur free tier
- Qualité variable Open Food Facts selon les produits
- Pas d'application iOS native packagée (faute de licence Apple Developer)

8.4 Perspectives

- Reconnaissance d'aliments par photo (vision IA) en complément du code-barres
- App iOS native via Expo EAS Build (licence Apple 99 €/an)
- Favoris et listes de courses
- Notifications push pour rappels nutritionnels
- Monétisation via abonnement premium
- Intégration applications sportives (Strava, MyFitnessPal)

8.5 Mot de clôture

Ce projet a permis à l'équipe de vivre un cycle de gestion de projet complet, de la charte initiale au déploiement cloud, en passant par conception, développement, qualité et livraison. Le partage du rôle de chef de projet a été particulièrement formateur en termes de responsabilisation et de compréhension globale des enjeux.

Au-delà du livrable technique, l'expérience nous a permis d'éprouver concrètement les concepts vus en cours : SMART, WBS, Gantt, plan qualité, matrice des risques, KPIs. Les écarts entre théorie de la charte et réalité du terrain (cascade vs itératif) ont été l'occasion de réflexions enrichissantes.

L'équipe remercie l'encadrement pédagogique, les intervenants professionnels du Crédit Agricole, et chacun de ses membres pour son engagement constant tout au long du semestre.

Alexandre, Asma, Augustin, Hassan, Sophie

17 Mai 2026